Tech Challenge SOAT

Fase 5 - Trabalho de Reposicao

Plataforma de Revenda de Veiculos Automotores

Entregaveis:

1. Desenho de Arquitetura (servicos serverless/gerenciaveis)
2. Relatorio de Seguranca de Dados
3. Relatorio de Orquestracao SAGA

Tecnologias: Python 3.12, Django 5.x/6 compativel, Django REST Framework

Deploy: Render (PaaS - Free Tier)

Banco: PostgreSQL (Render Managed)

Link do GitHub: https://github.com/MelloLipe/tech-fase-5-service-revenda-automotores

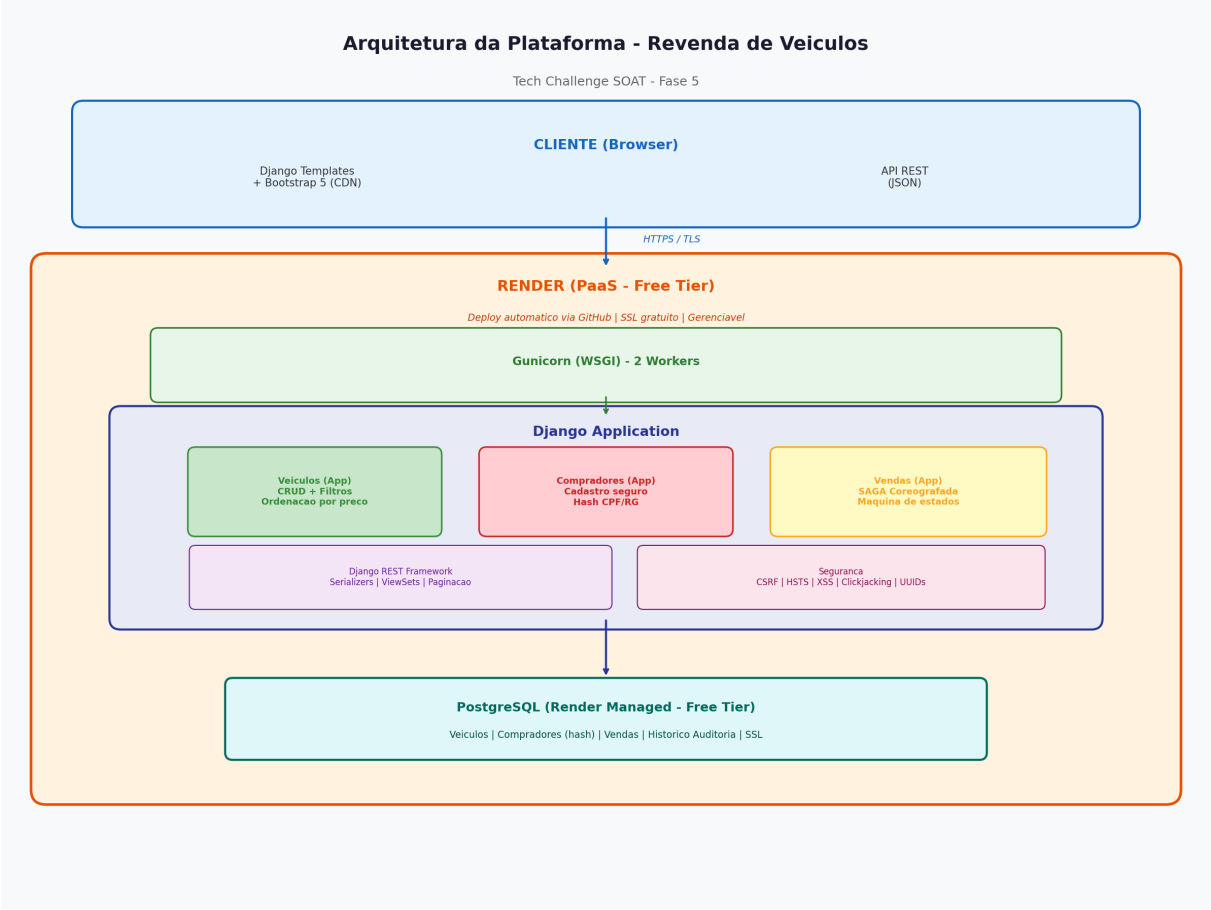
Link do video demonstrativo: https://drive.google.com/file/d/1iLkqwV-U346ZAoGVPaWvmSSsFPj_CQEt/view?usp=sharing

1. Desenho de Arquitetura

1.1 Visao Geral

A plataforma foi construida como uma aplicacao web utilizando Django + Django REST Framework, deployada no Render (PaaS gerenciavel na nuvem). A arquitetura prioriza servicos **serverless e gerenciaveis** para eliminar a necessidade de gerenciar infraestrutura, reduzir custos e garantir seguranca automatizada.

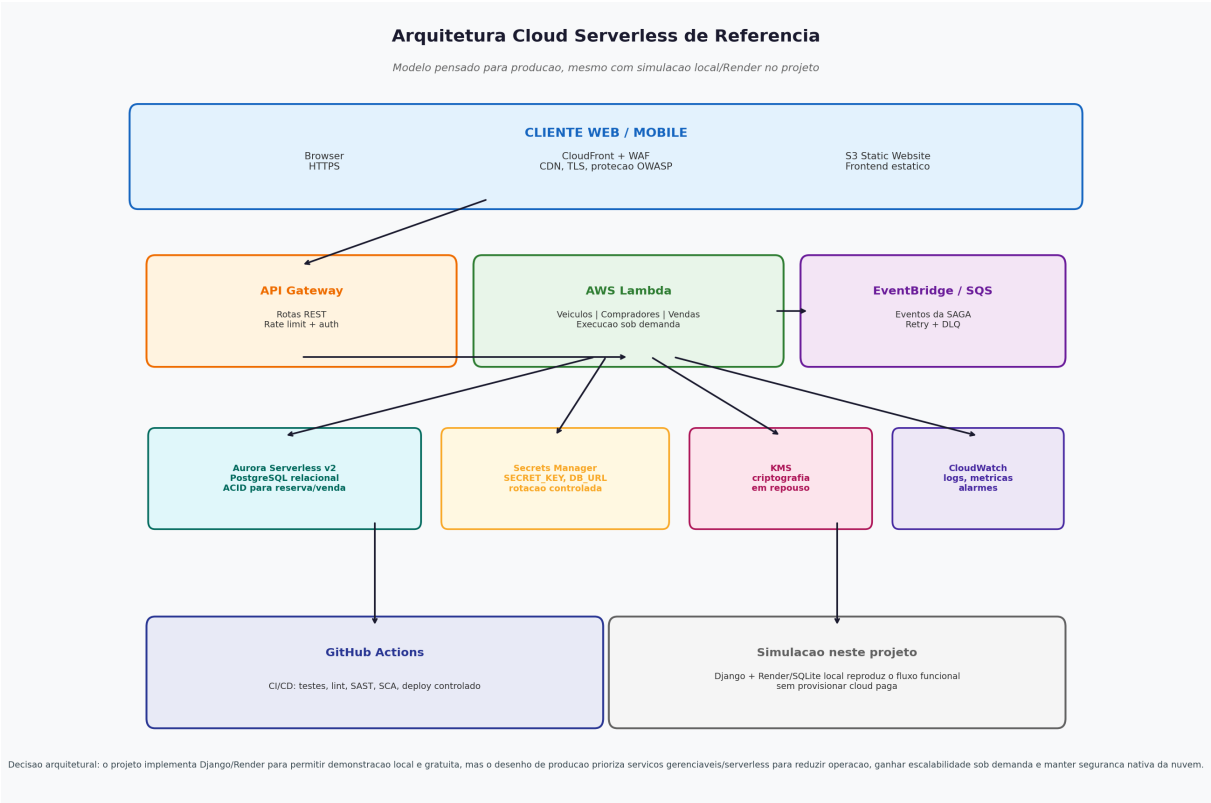
1.2 Diagrama de Arquitetura



O diagrama apresenta a solucao em 4 camadas: (1) Cliente/Browser acessando via HTTPS, (2) Render PaaS gerenciando o servidor Gunicorn, (3) Django Application com 3 apps (Veiculos, Compradores, Vendas/SAGA), e (4) PostgreSQL gerenciado pelo Render.

1.3 Arquitetura Cloud Serverless de Referencia

Embora a implementacao deste repositorio rode localmente e possa ser simulada no Render para fins academicos, a arquitetura foi pensada para uma evolucao em nuvem usando servicos serverless e gerenciaveis. O objetivo do desenho abaixo e demonstrar como a solucao poderia ser implantada em producao com menor operacao manual, escalabilidade sob demanda e seguranca nativa da plataforma cloud.



Neste modelo, CloudFront e WAF protegem e distribuem o frontend; API Gateway publica as rotas REST; AWS Lambda executa as regras de negocio sob demanda; EventBridge/SQS desacoplam eventos da SAGA; Aurora Serverless v2 preserva as transacoes relacionais necessarias para reserva e venda; Secrets Manager, KMS e CloudWatch concentram segredos, criptografia, logs e observabilidade.

1.4 Servicos Utilizados e Justificativas

Servico	Tipo	Justificativa
Render Web Service	PaaS Gerenciavel	Deploy automatico via GitHub, SSL gratuito, escalabilidade automatica. Elimina gerenciamento de servidores, patches e infraestrutura. Plano gratuito adequado para o projeto.
Render PostgreSQL	Banco Gerenciavel	Banco relacional robusto com transacoes ACID (essenciais para SAGA). Backups automaticos, conexao SSL, isolamento de rede. Suporte nativo a UUIDs. Gerenciado pela nuvem.
Render SSL/TLS	Seguranca na Nuvem	Certificados Let's Encrypt automaticos e gratuitos. Toda comunicacao criptografada sem configuracao manual. Previne man-in-the-middle e sniffing.
Django + DRF	Framework Web	Framework maduro com ORM, autenticacao integrada, admin automatico. Uma unica app serve API REST e frontend, economizando recursos. Middlewares de seguranca integrados.
Gunicorn	Servidor WSGI	Servidor de producao leve e eficiente. 2 workers para plano gratuito (512MB RAM). Pre-fork worker model para concorrencia.
WhiteNoise	Static Files	Serve arquivos estaticos sem necessidade de CDN ou Nginx separado. Compressao automatica (gzip). Zero custo adicional.

1.5 Alternativas Consideradas e Decisoes Arquiteturais

- **Render**: escolhido para simulacao executavel por ser simples, gratuito e gerenciavel, permitindo gravar o video e demonstrar a aplicacao sem provisionar cloud paga.

- **AWS Lambda + API Gateway**: arquitetura alvo serverless considerada para producao. Nao foi implementada no repositorio por aumentar complexidade, custo e tempo de entrega academica, mas foi modelada no desenho de referencia.

- **Aurora Serverless v2**: alternativa serverless ao PostgreSQL gerenciado, preservando SQL, transacoes ACID e escalabilidade automatica.

- **DynamoDB/NoSQL**: descartado como banco principal pela necessidade de transacoes relacionais e locks pessimistas no fluxo de reserva/venda, essenciais para consistencia da SAGA.

- **AWS ECS/Fargate**: alternativa gerenciavel para containers, mas com custo e operacao maiores que Render para o escopo do trabalho.

- **Heroku/Railway/Fly.io**: alternativas PaaS avaliadas, mas descartadas por custo, limites de free tier ou exigencia de cartao de credito.

1.6 Servicos de Seguranca na Nuvem

1.6.1 Render SSL/TLS (Seguranca de Transporte)

O que e: O Render fornece certificados SSL/TLS gratuitos e automaticos (Let's Encrypt) para todas as aplicacoes deployadas na plataforma.

Por que usamos: Garante que toda comunicacao entre cliente e servidor e criptografada, prevenindo ataques man-in-the-middle, sniffing de dados e interceptacao de credenciais. E um servico de seguranca gerenciado pela nuvem - nao requer configuracao manual de certificados, renovacao ou gerenciamento.

Configuracao: SECURE_SSL_REDIRECT = True redireciona automaticamente HTTP para HTTPS. HSTS com 1 ano de duracao forca o browser a sempre usar HTTPS.

Observacao: Em desenvolvimento local a aplicacao roda com DEBUG=True para facilitar testes. Em producao, o Render executa com DEBUG=False e HTTPS/TLS gerenciado pela plataforma.

1.6.2 Render Network Isolation (Seguranca de Rede)

O que e: O PostgreSQL no Render e acessivel apenas internamente, dentro da rede privada.

Por que usamos: O banco de dados nao tem IP publico, prevenindo ataques diretos. Apenas a aplicacao Django consegue se conectar, via conexao SSL interna.

1.6.3 Django Security Middleware (Seguranca de Aplicacao)

- **SecurityMiddleware (HSTS)**: Strict-Transport-Security forca HTTPS por 1 ano, prevenindo downgrade attacks.
- **CsrfViewMiddleware**: Token CSRF em cada formulario, prevenindo Cross-Site Request Forgery.
- **X-Content-Type-Options: nosniff**: Previne MIME-type sniffing pelo browser.
- **X-XSS-Protection**: Ativa filtro XSS nativo do browser.
- **X-Frame-Options: DENY**: Previne clickjacking (aplicacao embutida em iframes maliciosos).
- **Cookies Secure + HttpOnly**: Cookies de sessao so transmitidos via HTTPS e inacessiveis via JavaScript.

Justificativa: Sao protecoes padrao da industria contra os ataques web mais comuns (OWASP Top 10). O Django fornece essas protecoes integradas, bastando habilita-las na configuracao.

1.6.4 Hashing Criptografico de Dados Sensiveis

O que e: CPF e RG sao tratados como dados pessoais identificadores criticos e armazenados como hashes SHA-256 (funcao de hash criptografica irreversivel). Apenas o CPF mascarado (***.***.*XX-XX) e visivel na interface.

Por que usamos: Mesmo em caso de vazamento completo do banco de dados, e computacionalmente inviavel recuperar os documentos originais a partir dos hashes. Essa e uma medida de protecao alinhada a LGPD para reduzir exposicao e risco de fraude sobre dados pessoais identificadores.

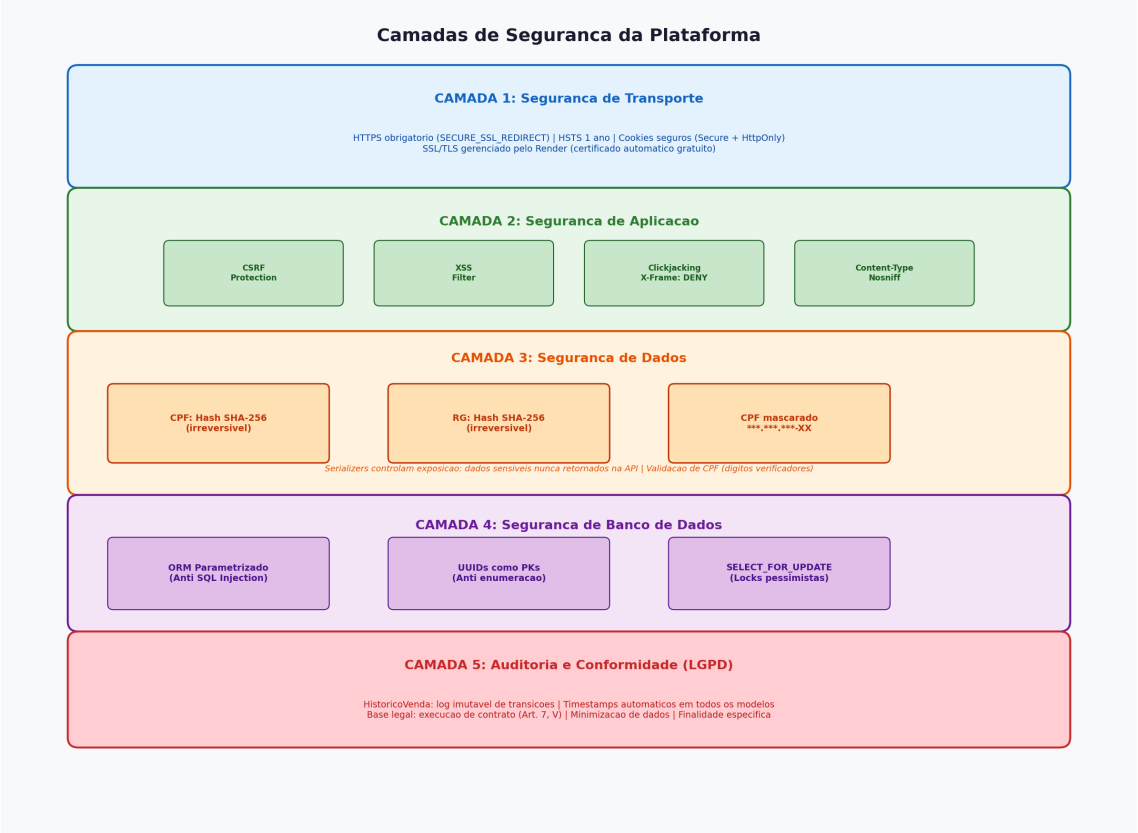
1.6.5 Variaveis de Ambiente (12-Factor App)

O que e: SECRET_KEY, DATABASE_URL e configuracoes sensiveis sao gerenciadas via variaveis de ambiente (os.environ), nunca armazenadas no codigo-fonte.

Por que usamos: Seguimos o principio dos 12-Factor Apps. Nenhum segredo e exposto em repositorios publicos. O Render gerencia as variaveis de ambiente de forma segura.

2. Relatorio de Seguranca de Dados

2.1 Visao Geral das Camadas de Seguranca



A plataforma implementa 5 camadas de seguranca independentes e complementares, desde o transporte de dados ate a conformidade com a LGPD.

2.2 Dados Armazenados pela Solucao

2.2.1 Dados de Veiculos

Campo	Tipo	Sensibilidade
Marca, Modelo, Ano, Cor	Texto	Publica
Preco	Decimal	Publica
Placa	Texto (unique)	Moderada
Chassi	Texto (unique)	Moderada
Quilometragem	Inteiro	Publica
Status	Enum	Publica

2.2.2 Dados de Compradores

Campo	Tipo	Sensibilidade	Tratamento
Nome	Texto	Pessoal (LGPD)	Texto plano
Email	Texto	Pessoal (LGPD)	Texto plano
Telefone	Texto	Pessoal (LGPD)	Texto plano
CPF	**Hash SHA-256**	**Identificador critico**	**Hash irreversivel + mascaramento**
RG	**Hash SHA-256**	**Identificador critico**	**Hash irreversivel**
Endereco	Texto	Pessoal (LGPD)	Texto plano
Data Nascimento	Data	Pessoal (LGPD)	Texto plano

2.2.3 Dados de Vendas

Campo	Tipo	Sensibilidade
Codigo de Pagamento	UUID	Confidencial
Preco de Venda	Decimal	Comercial
Status da Venda	Enum	Operacional
Timestamps	DateTime	Operacional
Historico/Auditoria	Texto	Operacional

2.3 Dados Sensiveis Identificados

Dados pessoais identificadores de alta criticidade:

- **CPF**: Documento de identificacao fiscal. Armazenado como hash SHA-256 (irreversivel). Apenas os 4 ultimos digitos sao visiveis na interface (mascaramento: ***.***.*XX-XX).

- **RG**: Documento de identificacao civil. Armazenado como hash SHA-256 (irreversivel). Nunca exibido na interface ou retornado pela API.

Dados pessoais (LGPD Art. 5, I): Nome, email, telefone, endereco, data de nascimento, CPF e RG permitem identificacao do titular. CPF e RG foram tratados com controles reforcados por risco de fraude e dano ao titular.

2.4 Politicas de Acesso a Dados Implementadas

- **Principio do Menor Privilegio**: A API de leitura nunca expoe CPF ou RG em texto plano. Apenas o CPF mascarado e retornado. O hash completo nunca e exposto.

- **Serializers com Controle de Exposicao**: CompradorCreateSerializer aceita CPF/RG em texto plano (write-only) e gera hashes imediatamente. CompradorSerializer retorna apenas CPF mascarado.

- **Admin Django com Restricoes**: Campos sensiveis (cpf_hash, rg_hash) sao readonly e nao editaveis, mesmo para administradores.

- **Validacao de Dados na Entrada**: CPF e validado algoritmicamente (digitos verificadores) antes do armazenamento. RG e validado por formato minimo. Email validado por formato e unicidade.

- **UUIDs como Chaves Primarias**: Previne enumeracao sequencial de recursos (ataques IDOR). Impossivel adivinhar IDs de outros compradores ou vendas.

2.5 Politicas de Seguranca da Operacao Implementadas

Seguranca de Transporte:

- HTTPS obrigatorio: SECURE_SSL_REDIRECT = True em producao.

- HSTS: SECURE_HSTS_SECONDS = 31536000 (1 ano), incluindo subdominios e preload.

- Cookies seguros: SESSION_COOKIE_SECURE = True, CSRF_COOKIE_SECURE = True.

- Ambiente local usa DEBUG=True apenas para demonstracao; ambiente de producao usa DEBUG=False.

Seguranca de Aplicacao:

- CSRF Protection: Token CSRF em todos os formularios.

- XSS Protection: SECURE_BROWSER_XSS_FILTER = True.

- Content-Type Sniffing: SECURE_CONTENT_TYPE_NOSNIFF = True.

- Clickjacking: X_FRAME_OPTIONS = DENY.

- Secret Key: Gerada automaticamente via variavel de ambiente, nunca no codigo.

Seguranca de Banco de Dados:

- Conexao criptografada: PostgreSQL via SSL no Render.

- ORM parametrizado: Prevencao de SQL Injection via Django ORM. Nenhuma query raw.

- SELECT_FOR_UPDATE: Locks pessimistas para operacoes concorrentes (reservas de veiculos).
- Transacoes atomicas: transaction.atomic() garante consistencia em cada etapa da SAGA.

Auditoria:

- HistoricoVenda: Log imutavel de todas as transicoes de estado (SAGA).
- Timestamps automaticos: criado_em e atualizado_em em todos os modelos.
- Rastreabilidade completa do fluxo de cada venda, incluindo compensacoes.

2.6 Riscos e Acoes de Mitigacao

Risco	Impacto	Descricao	Mitigacao
Vazamento de Banco de Dados	Alto	Dados pessoais expostos	CPF e RG armazenados como hash SHA-256 irreversivel. Mesmo com acesso ao banco, nao e possivel recuperar os documentos originais. CPF mascarado na interface.
Condicao de Corrida na Reserva	Medio	Dois clientes reservam o mesmo veiculo	SELECT_FOR_UPDATE com locks pessimistas. Transacoes atomicas (transaction.atomic()). Verificacao de status do veiculo antes de cada operacao.
Reserva Sem Pagamento	Medio	Veiculo fica indisponivel indefinidamente	Expiracao automatica de reservas em 30 minutos. Compensacao SAGA libera o veiculo.
Acesso Nao Autorizado a API	Alto	Manipulacao de dados	Para producao: implementar autenticacao JWT ou OAuth2 via DRF. Rate limiting para prevenir abuso.
Forca Bruta em CPF	Medio	CPFs tem espaco limitado (11 digitos)	Hash SHA-256 reduz exposicao direta. Para producao: adicionar segredo/salt com HMAC ou algoritmo resistente como bcrypt/argon2 para reduzir ataques por dicionario.
Injecao de SQL	Critico	Acesso/modificacao de dados	Django ORM com queries parametrizadas. Nenhuma query SQL raw utilizada.

2.7 Conformidade LGPD

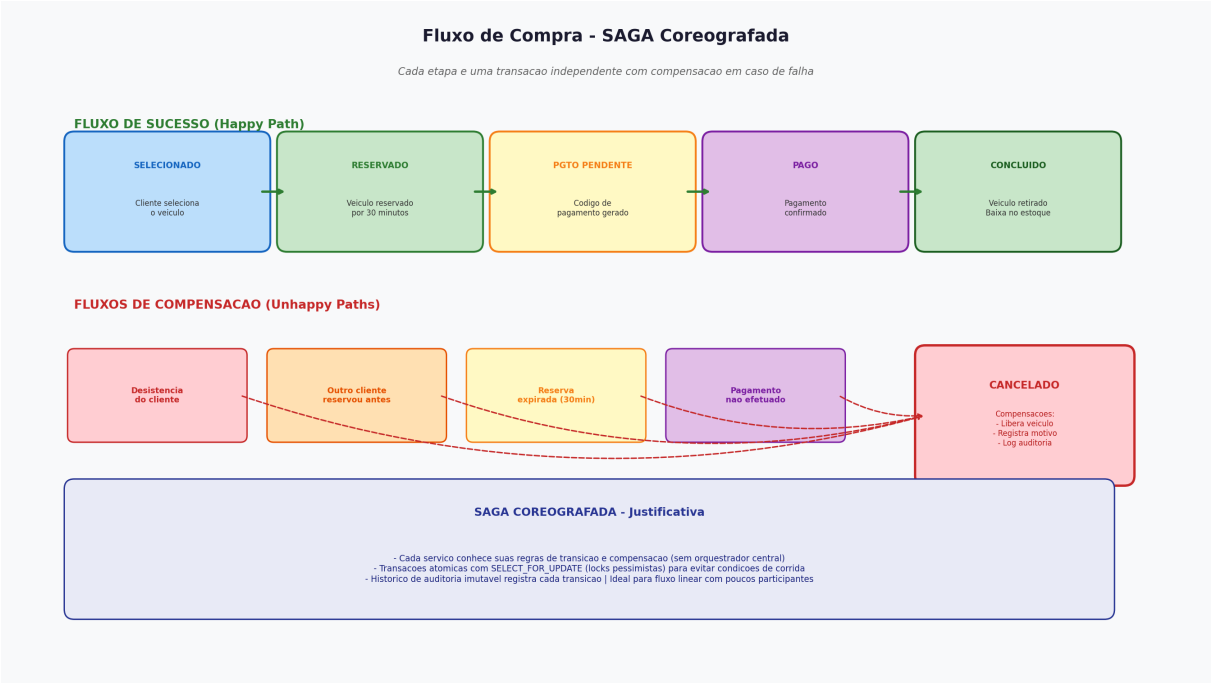
- **Base Legal**: Execucao de contrato (Art. 7, V) - dados necessarios para processo de compra e documentacao veicular.
- **Minimizacao**: Apenas dados estritamente necessarios sao coletados.
- **Finalidade**: Dados utilizados exclusivamente para o processo de compra e documentacao.
- **Transparencia**: CPF mascarado indica ao titular que seus dados estao protegidos.
- **Seguranca**: Medidas tecnicas de protecao implementadas (hashing, HTTPS, headers).

3. Relatorio de Orquestracao SAGA

3.1 Tipo de SAGA Escolhido: Coreografia (Choreography)

Para este projeto, foi escolhida a **SAGA Coreografada** ao inves da SAGA Orquestrada.

3.2 Diagrama do Fluxo SAGA



3.3 Justificativa da Escolha

Razoes Tecnicas:

- **Arquitetura Monolitica**: Como a aplicacao e um monolito Django (nao microsservicos), nao ha necessidade de um orquestrador central separado. Cada servico (service layer) conhece suas proprias regras de transicao e compensacao.

- **Simplicidade**: A coreografia e mais simples de implementar em aplicacoes monoliticas, onde todas as transacoes ocorrem no mesmo banco de dados.

- **Economia de Recursos**: Nao requer um servico de orquestracao separado (como RabbitMQ, Kafka), reduzindo custos de infraestrutura - essencial para deploy em plano gratuito.

- **Acoplamento Baixo**: Cada etapa do processo e independente e contem sua propria logica de compensacao, facilitando manutencao e testes.

Razoes de Negocio:

- **Fluxo Linear**: O processo de compra e essencialmente linear (selecionar -> reservar -> pagar -> retirar), sem ramificacoes complexas.

- **Poucos Participantes**: Apenas 3 entidades participam (Veiculo, Comprador, Venda), tornando a coreografia gerenciavel.

- **Compensacoes Simples**: As acoes compensatorias sao diretas (liberar reserva, cancelar venda), sem necessidade de coordenacao complexa.

3.4 Fluxo Detalhado da SAGA

Fluxo de Sucesso (Happy Path):

1. **SELECIONADO**: Cliente seleciona o veiculo desejado. Sistema verifica disponibilidade.
2. **RESERVADO**: Veiculo e reservado por 30 minutos com lock pessimista (SELECT_FOR_UPDATE).
3. **PAGAMENTO PENDENTE**: Codigo de pagamento (UUID) e gerado e enviado ao cliente.
4. **PAGO**: Cliente confirma pagamento com o codigo recebido. Sistema valida.
5. **CONCLUIDO**: Veiculo e retirado pelo cliente. Status do veiculo muda para VENDIDO.

Cenarios de Compensacao:

Cenario	Etapa	Compensacao
Veiculo ja reservado por outro	Reserva	Cancela venda automaticamente, registra motivo no historico
Reserva expirada (>30 min)	Pagamento	Libera veiculo, cancela venda, registra no historico
Pagamento nao efetuado	Pagamento	Libera veiculo, cancela venda
Cliente desiste	Qualquer	Libera veiculo (se reservado), cancela venda, registra motivo
Codigo pagamento invalido	Confirmacao	Rejeita operacao, mantem status anterior

3.5 Implementacao Tecnica

- **Transacoes Atomicas**: Cada etapa da SAGA e envolvida em transaction.atomic() com SELECT_FOR_UPDATE para garantir atomicidade, isolamento e consistencia.

- **Historico de Auditoria**: Cada transicao e registrada na tabela HistoricoVenda (status anterior, novo, descricao, timestamp), permitindo rastreabilidade completa.

- **Service Layer**: Logica de negocio isolada em vendas/services.py com funcoes especificas para cada etapa e compensacao.

- **Idempotencia**: Cada operacao verifica o status atual antes de executar, garantindo que transicoes invalidas sejam rejeitadas.

3.6 Comparacao: Coreografia vs. Orquestracao

Aspecto	Coreografia (Escolhida)	Orquestracao
Complexidade	Baixa	Alta
Infraestrutura	Nenhuma adicional	Message broker necessario
Custo	Zero adicional	Servidor de orquestracao
Acoplamento	Baixo	Centralizado
Rastreabilidade	Via logs de auditoria	Via orquestrador
Adequacao	Ideal (fluxo linear)	Excessivo para o escopo